

2025 SP Assignment #1

Due: Oct. 22nd 12:00pm

Last updated: 2025-09-30

1. 과제 개요

리눅스 커널은 **커널 모듈(Kernel Module)**이라는 메커니즘을 통해, 실행 중인 커널을 재컴파일하거나 재부팅하지 않고도 동적으로 기능을 추가하거나 제거할 수 있는 인터페이스를 제공한다. 또한, **proc 가상 파일 시스템**은 커널 내부의 다양한 데이터 구조와 통계 정보를 사용자 공간(User Space)에 텍스트 형태로 손쉽게 전달하는 통로 역할을 한다.

이번 과제에서는 **커널 모듈 프로그래밍**을 통해 커널의 핵심 데이터 구조인 프로세스와 스케줄러의 내부 상태를 직접 조회할 수 있는 커널 모듈을 구현하고, 이를 proc 가상 파일 시스템을 통해서 확인해본다. 이를 통해 수업시간에 배운 프로세스와 스케줄러가 리눅스에 어떻게 구현되어 있는지를 실제로 경험해보는 것을 목표로 한다. 과제는 보고서 과제와 실습 과제로 이루어진다.

2. 배경 지식: 커널 모듈 프로그래밍

이번 실습 과제는 커널 모듈 프로그래밍을 통하여 원하는 커널 정보를 proc 파일 시스템을 통해 확인하는 커널 모듈을 개발하는 것이다. 커널 모듈 프로그램은 리눅스 커널 코드를 직접 수정하고 컴파일 하지 않고도 필요할 때 동적으로 커널에 추가되어 동작할 수 있다. 커널 모듈 프로그램은 커널의 일부로 동작하기에 커널의 동작에 관여하여 커널 내부 자료구조에 접근할 수 있다. proc 파일 시스템은 리눅스의 가상 파일 시스템 중 하나로, 이를 통해 사용자가 커널의 정보를 쉽게 얻을 수 있도록 한다. 예를 들어, 사용자가 터미널에 `cat /proc/meminfo` 를 입력하면 시스템의 메모리 정보를 얻을 수 있게 된다. 이번 과제에서도 이를 이용하여 필요한 시스템 정보를 얻는다. 이러한 proc 파일 시스템은 커널 모듈 프로그래밍을 통해 구현할 수 있다. 이번 과제에서는 커널 코드를 직접 수정하지 않는다.

사용자가 `cat /proc/hello` 를 터미널에 입력하였을 때, "Hello, Kernel!"라는 메시지를 띄우도록 하는 커널 모듈 프로그램의 예시 코드는 아래와 같다. 이번 과제를 할 때에는 아래의 예시 코드를 참고해서 진행하면 된다. 커널 모듈 프로그램은 gcc 컴파일러와 표준 라이브러리 대신에 별도의 컴파일러와 헤더를 이용해야 하는 것에 주의해야 한다. 즉, 커널 모듈을 위한 c 언어 규칙은 gcc 컴파일러를 기준으로 작성하는 일반 c 언어와 다른 부분이 있다. Makefile 규칙 역시 다소 다른데, 위의 링크에서 함께 확인할 수 있다.

커널 모듈은 터미널에서 `sudo insmod` 명령어를 통해 커널에 등록되는데, 이때 `module_init()`을 통해 등록된 함수가 실행된다. 커널 모듈은 터미널에서 `sudo rmmod` 명령어를 통해 커널에서 제거되는데, `module_exit()`을 통해 등록된 함수가 실행된다. 커널 모듈 프로그래밍을 할 때에는 이러한 기본 양식을 지켜야 한다. 커널 모듈 프로그래밍을 할 때, 결과 출력은 예시 코드처럼 `seq_printf()` 함수를 이용하는 것을 추천한다. 또한 proc 파일 시스템에 생성을 위하여 `proc_ops` 구조체를 사용한다. 자세한 사용 방법은 첨부한 커널 모듈 프로그래밍 가이드 사이트를 참고한다.

3. 보고서 과제

3.1 보고서 양식

보고서는 **사전 조사 보고서**와 **프로그래밍 수행 결과 보고서**로 구성한다. 이번 과제의 보고서 양식은 **첨부한 ACM 양식에 따라** 작성해야 한다. 해당 양식은 실제 학술 논문에서 사용되는 표준 형식으로 논리적 흐름, 실험 설계 및 결과 해석 등을 구조화하는데 도움이 된다. 다음은 상세 보고서 양식 사항이다.

- 보고서 제목은 SP Assignment 1 Report 로 한다.
- 보고서 Section 1 에는 사전 조사 보고서를, 이후 Section 에는 프로그래밍 수행 결과 보고서로 구성한다.
- 각 Section title(1, 2, 3, ...)은 10pt 로, 각 Section subtitle(1.1, 1.2, 2.1, ...) 및 본문은 9pt 로 한다.
- 영어는 Linux Libertine O, 한글은 나눔스퀘어 폰트로 한다.
- 언어는 원하는 언어를 사용한다.
- 이외의 양식은 template 에 서술되어 있는 양식을 따른다.

3.2 보고서 내용

- 사전 조사 보고서
 - 다음 두 버전의 리눅스 커널 코드를 비교 분석한다.
 - 리눅스 커널 2.6.39 버전의 스케줄러 관련 코드
 - 리눅스 커널 6.14 버전의 스케줄러 관련 코드
 - 각 버전의 스케줄링 관련 커널 자료구조에 대해 비교 분석한다.
 - cfs_rq, sched_entity, task_struct 를 중심으로 분석한다
 - 각 커널 자료구조의 멤버 변수들 중에서 스케줄링 관점에서 중요하거나 실습 과제와 관련이 있는 멤버 변수를 위주로 분석한다.
 - 각 버전의 공통점과 차이점을 중심으로 비교 분석한다.
 - CFS 스케줄러가 Red-black 트리에서 어떻게 다음 프로세스를 선택하는지 분석한다.
 - L02 강의 노트를 참고하여 작성한다.
 - 코드 분석에 필요한 리눅스 커널 코드는 아래 링크에서 확인할 수 있다.
 - <https://elixir.bootlin.com/linux/v6.17/source>
 - 순서도나 그림 등을 활용해 정리한다. (3 개 이상)
 - **참조 문헌**
- 프로그래밍 수행 결과 보고서
 - 작성한 커널 모듈의 전체 구조 및 동작 과정
 - 실습 과제에서 구현한 2 개의 모듈에 대해 각각 정리하여 서술
 - **순서도나 블록 다이어그램 등 도식화 자료를 반드시 이용할 것**
 - 자세한 코드 설명 보다는 전체적인 flow 를 중점으로 서술

- 개발 환경 명시
 - `uname -a` 실행 결과
 - 사용한 컴파일러 버전, 메모리 정보 등
- 과제 수행 중 발생한 애로사항 및 해결 방법

4. 실습 과제

실습 과제는 총 2 개의 커널 모듈을 작성하는 것이다. 실습 과제를 위한 준비사항과 각 모듈에 대한 상세 설명은 다음과 같다.

4.1 준비 사항

- 실습 과제 진행 중에 커널이 깨지는 경우가 발생할 수 있기에 VirtualBox 나 VMWare 와 같은 가상머신 환경에서 과제를 진행하는 것을 권장한다. 단, 가상 머신의 CPU 코어의 수는 반드시 2 개 이상으로 설정해야한다.
- 64-bit PC 용 이미지 Ubuntu-24.04.3 을 다운 받아 가상 머신 환경에 설치한다. 실습 과제 채점은 제출한 프로그램이 우분투 24.04.3 버전에서 정상적으로 동작하는지가 기준임에 유의한다. 우분투 24.04.3 을 설치하면 설치된 커널 버전은 6.14.0(6.14.0-32-generic)일 것이다(`uname -a`로 확인 가능). 만약 커널 버전이 6.14.0 이 아니라면 커널의 버전을 6.14.0 으로 변경해야 한다. 아래 링크에서 해당 커널을 다운받을 수 있다.
 - <https://releases.ubuntu.com/noble/>
 - <https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.14.tar.gz>

4.2 PART 1 – ancestor

이 모듈은 특정 PID 를 입력받아 해당 프로세스의 모든 조상 프로세스 정보를 출력한다. 커널 모듈이 생성하는 proc 파일 이름은 "ancestor"로 한다.

- 터미널에서 다음 명령어를 통해 원하는 정보를 확인할 수 있다.

```
$ cat /proc/ancestor
```

- 모듈은 `/proc/ancestor` 라는 읽기/쓰기가 가능한 파일을 생성해야 한다.
 - Write 동작: `echo <PID> > /proc/ancestors` 명령을 통해 사용자가 입력한 PID 를 모듈 내부에 저장해야 한다.
 - Read 동작: `cat /proc/ancestors` 명령 시, 저장된 PID 의 프로세스부터 시작하여 모든 조상의 정보를 한 줄씩 출력한다.
- 출력 양식 및 출력 예시는 다음과 같다.
 - 먼저, 기본 정보를 상단에 출력한다.

- 학번, 이름
- 이후 지정한 PID 프로세스부터 한줄씩 조상 프로세스를 출력한다.
- **출력 양식이 다를 경우 감점이 있을 수 있으니 출력 양식에 주의한다.**
 - 기본 정보와 프로세스 정보 사이의 '-' 개수는 40 개이다.

```
// 출력 양식: [PID] 프로세스이름
```

```
ID: 2025123456
```

```
Name: Hong, Gildong
```

```
-----
```

```
[5432] bash
```

```
[5401] gnome-terminal
```

```
[1] systemd
```

4.3 PART 2 - proc_analyzer

이 모듈은 특정 프로세스의 PID 를 입력받아, 해당 프로세스와 그 아래에 속한 모든 자손 프로세스들 중에서 CFS runqueue 에 존재하는 프로세스들에 대해서만 상세한 스케줄링 정보를 CPU 별로 나누어 출력한다.

- 모듈은 /proc/proc_analyzer 라는 읽기/쓰기가 가능한 파일을 생성해야 한다.
 - Write 동작: echo <PID> > /proc/proc_analyzer 명령을 통해 사용자가 입력한 PID 를 모듈 내부에 저장해야 한다.
 - Read 동작: cat /proc/proc_analyzer 명령 시, 저장된 PID 와 그 모든 자손들을 탐색한다. 그 중에서 현재 CFS runqueue 에서 실행 대기 중인 프로세스들만 CPU 별로 그룹화하고, **각 그룹 내에서 vruntime 오름차순으로 출력한다.**
- 출력 양식 및 출력 예시는 다음과 같다. (아래의 예시는 임의의 값을 유의한다.)
 - 먼저, PART 1 에서와 동일하게 기본 정보를 상단에 출력한다.
 - 추가적으로 예시에서와 같이 입력받은 PID 를 출력한다.
 - 이후 각 CPU 마다 CFS runqueue 에 있는 프로세스들의 개수와 프로세스들을 vruntime 오름차순으로 출력한다.
 - CPU 번호는 0 번부터 시작하며, **오름차순으로 출력한다.**
- 특별한 프로그램이 실행되지 않는 우분투 환경에서는 일반적으로 그 순간에 CFS runqueue 에 등록된 프로세스의 수가 한두개 밖에 없다. 따라서 여러 개의 프로세스를 생성, 동작하는 임의의 테스트 프로그램을 구현하고 실행하여 과제 구현물을 테스트해볼 수 있다.
- 커널의 데이터에 접근할 때에는 적절한 lock 이 필요할 수 있다.

```
// 출력 양식: [PID] 프로세스이름 vruntime
ID: 2025123456
Name: Hong, Gildong
PID: 3734
```

```
-----
[CPU #0] Running processes: 2
[3286] python3 123456439045
[3290] python3 123456462280
-----
```

```
-----
[CPU #1] Running processes: 3
[3391] cat 123457185643
[3280] python3 123458242690
[3289] python3 124578976015
```

4.4 실습 과제 컴파일 조건

실습 과제는 **하나의 Makefile 을 통해 컴파일 되어야 한다.** 즉, make 빌드 시스템으로 빌드를 하게 되면 make 명령어를 실행한 그 경로에 ancestor.ko, proc_analyzer.ko 의 커널 모듈 파일이 생성되어야 한다. make clean 을 하게 되면 빌드 결과로 생성된 파일들이 모두 삭제되어야 한다. 생성된 커널 모듈은 sudo insmod 명령어로 등록되어야 하며 sudo rmmod 명령어를 통해 삭제할 수 있어야 한다.

(예시)

```
os@ubuntu:~$ cd 2025123123
os@ubuntu:~/2025123123$ make
os@ubuntu:~/2025123123$ ls
Makefile ancestor.ko proc_analyzer.ko ...
os@ubuntu:~/2025123123$ make clean
Makefile ...
```

5. 과제 제출

과제는 보고서 pdf 파일 (hw1_[학번].pdf)과 소스 압축 파일 (hw1_[학번].tar)을 제출한다. 보고서 파일의 경우 반드시 PDF 파일로 변환하여 제출하며 파일의 이름은 반드시 hw1_[학번].pdf 로 한다. 실습 과제 소스 파일의 경우 반드시 다음 명령어를 사용해서 압축 파일을 생성한다. **다른 형식의 압축 또는 이중 압축은 절대 허용하지 않으며 보고서 파일을 실습 과제 파일과 함께 압축해서는 안된다.**

```
# 본인이 작업하는 디렉토리가 ~/hw1 라고 가정하자.
이는 Makefile 이 ~/hw1/ 에 있음을 의미한다. (vi ~/hw1/Makefile)
# 본인의 학번이 2025123123 이라고 가정하면, 아래의 일련의 명령어를 수행한 이 후에
hw2_2025123123.tar 을 제출한다.
# 압축을 해제하였을 때 본인의 학번으로 된 폴더 하나가 나타나야 하고, 그 폴더 바로 아래에 Makefile 이 있어야
함을 의미한다.
$ cd ~/hw1/..
```

```
$ mv hw1 2025123123
$ tar cvf hw1_2025123123.tar 2025123123
```

6. 주의 사항

- 채점 환경에서 컴파일되지 않으면 실습 과제가 0 점이 되므로 리눅스 배포판 버전, 파일 인코딩, 커널 버전 등에 주의한다.
- CPU 의 개수는 2 개 이상으로 임의로 정해지기 때문에 다양한 CPU 환경에서 동작 가능해야 한다.
- **적절한 사유 없이 Delay 될 경우 절대 받지 않음**
- 그림, 코드 조각을 포함한 모든 참고 자료 내용은 **반드시 출처를 명시 (출처 명시 안했을 시 감점)**
- 타 수강생의 보고서 및 과제를 카피 또는 수정하여 제출하였을 경우 모두 0 점 처리
- 제출 파일 생성 방법과 프로그램 구조(**특히 프로그램의 이름**)를 지키지 않을 경우 감점 또는 0 점 처리
- 제출한 소스에는 **반드시 주석을 달 것**, 주석의 내용이 불분명하거나 없을 경우 감점 처리
- 제출한 과제가 컴파일이 되지 않는 경우 0 점 처리
- 과제에 대한 문의 사항이 있을 경우 LearnUs 의 Q&A 게시판에 **'공개 게시글'**로 문의

7. 참고문헌

- <https://elixir.bootlin.com/linux/v6.14/source>
- <https://sysprog21.github.io/lkmpg/>
- <https://docs.kernel.org/core-api/index.html>